PROJECT CODE REPORT

FAQ Chatbot System

using LangChain

YOLOv8 | FAISS | HuggingFace | Streamlit | OpenRouter

Submitted By

Wajiha Adnan DT-23001,
FizzaAhmed DT-23304,
M.Abdullah Kidwai
DT-23302,
Taaha Nadeem DT-23303

2025

1. Introduction

In today's fast-paced digital world, businesses receive a large number of frequently asked questions from customers every day. Answering each question manually takes time and effort. A FAQ Chatbot System solves this problem by automatically reading and understanding company documents, and then answering user questions intelligently — without human involvement.

This project builds a complete FAQ Chatbot System using LangChain, a popular Python framework for building AI-powered applications. The chatbot is capable of reading PDF files and text documents, understanding their content, and providing accurate answers to user questions through a simple and friendly web interface built with Streamlit.

The system combines several modern AI technologies — document loading, text splitting, vector embeddings, a FAISS database, and a Large Language Model (LLM) — to create a smart, reliable, and easy-to-use chatbot solution.

2. System Overview

The FAQ Chatbot System works as a pipeline — a series of connected steps where the output of one step becomes the input for the next. The overall goal is to take raw documents and turn them into a smart, interactive chatbot that can answer questions based on what is written in those documents.

The system is made up of four main Python files, each handling a specific responsibility:

File	Responsibility
preprocessing.py	Loads documents (PDFs + text files) and splits them into smaller pieces
model.py	Creates embeddings, stores them in a FAISS vector database, and sets up the LLM
chatbot.py	Designs the prompt, retrieves relevant content, and builds the QA chain
app.py	Provides the Streamlit web interface where users interact with the chatbot

Together, these four modules form a complete Retrieval-Augmented Generation (RAG) system — a modern AI approach where the model answers questions based on real retrieved documents rather than from general knowledge alone.

3. Module-wise Explanation

3.1 preprocessing.py — Document Loading & Chunking

This is the first module in the pipeline. Its job is to load all the source documents and prepare them for further processing.

The module loads two types of documents: PDF files (such as terms and conditions, privacy policies) using PyPDFLoader, and plain text files (such as emails and chat logs) using TextLoader. Once all documents are loaded, they are combined into a single collection.

Since documents can be very long, they are split into smaller pieces called chunks using RecursiveCharacterTextSplitter. Each chunk is 500 characters long with a 50-character overlap between adjacent chunks. This overlap ensures that no important information is lost at the boundary between two chunks.

At the end of this module, the system has a list of small, manageable text chunks that are ready to be converted into embeddings.

preprocessing.py

C: > Users > wajih > AppData > Local > Temp > 55680994-8e89-4ee9-b9c6-fc53adc8dfbf_DT-23001_CCP Project.zip.fbf > CCP_FAQ_Chatbot > preprocessing.py

```python
from langchain_community.document_loaders import PyPDFLoader, TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

files = ["data/terms.pdf", "data/privacy.pdf"]
docs = []

for file in files:
    loader = PyPDFLoader(file)
    docs.extend(loader.load())

text_files = ["data/emails.txt", "data/chat_logs.txt"]

for file in text_files:
    loader = TextLoader(file)
    docs.extend(loader.load())

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_documents(docs)

print("Total chunks:", len(chunks))
```

3.2 model.py — Embeddings, Vector Database & LLM

This module is responsible for the intelligence behind the chatbot. It handles three important tasks: creating embeddings, building the vector database, and setting up the language model.

Embeddings are numerical representations of text that capture the meaning of each chunk. The system uses the all-MiniLM-L6-v2 model from HuggingFace to generate these embeddings. Once created, all chunk embeddings are stored in a FAISS vector database — a fast and efficient tool for searching similar text by meaning, not just keywords.

For answering questions, the module uses ChatOpenRouter connected to a powerful GPT-class language model. The LLM is configured with a low temperature (0.3) to keep answers focused and factual, and a token limit of 512 to control response length. An API key is loaded securely from an environment variable using python-dotenv.

C: > Users > wajih > AppData > Local > Temp > 5ba4e205-be1b-4135-b368-0d9a4677bd70_DT-230

```python
import os
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_openrouter import ChatOpenRouter
from dotenv import load_dotenv

load_dotenv()

# Lightweight embedding model - replaces TensorFlow/USE
embeddings = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"
)

# Load OpenRouter API key from environment
openrouter_api_key = os.getenv("OPENROUTER_API_KEY")

# Initialize OpenRouter model
local_llm = ChatOpenRouter(
    model="openai/gpt-oss-120b:free",
    temperature=0.3,
    max_tokens=512,
    openrouter_api_key=openrouter_api_key
)

def build_vectorstore(chunks):
    vectorstore = FAISS.from_documents(chunks, embeddings)
    print("Vector database created successfully")
    return vectorstore
```

3.3 chatbot.py — Prompt Design, Retrieval & QA Chain

This module is the brain of the chatbot. It connects all the pieces together and defines how the chatbot actually answers a question.

A custom system prompt is defined to tell the LLM to behave as an expert customer support document analyst. This ensures the chatbot stays focused on the documents and does not make up answers from general knowledge.

The module also handles smart retrieval logic. When a user asks a question, the system first checks whether the question is related to emails or messages using a keyword list. Based on this, it decides how to search the vector database for the most relevant chunks. A helper function then formats the retrieved chunks into a clean block of text that is passed to the LLM.

Finally, the QA chain is built using LangChain's RunnablePassthrough and StrOutputParser components. This chain takes the user's question, retrieves relevant context, constructs the full prompt, sends it to the LLM, and returns a clean text response.

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

system_prompt = (
    "You are an expert customer support document analyst.\n\n"
    "You have access to company documents including PDFs, emails, and chat logs.\n\n"
    "STRICT RULES:\n"
    "1. Read ALL the provided context carefully.\n"
    "2. If asked about emails — list EVERY email subject you find in the context, numbered.\n"
    "3. If asked to count something — count exactly what appears in the context.\n"
    "4. If asked to list something — list every item found, numbered.\n"
    "5. Be specific: extract names, subjects, dates, numbers directly from context.\n"
    "6. If partial info exists — share it and say it may be incomplete.\n"
    "7. NEVER say not available if the context has anything even slightly related.\n"
    "8. Only say 'not available' if context has zero relevance to the question.\n\n"
    "Context:\n{context}"
)

prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}"),
])

def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

EMAIL_KEYWORDS = [
    "email", "emails", "subject", "subjects", "mail",
    "received", "inbox", "message", "messages", "how many emails"
]

def build_qa_chain(vectorstore, local_llm, all_chunks=None):

    def smart_retriever(query):
        query_lower = query.lower()
        is_email_query = any(kw in query_lower for kw in EMAIL_KEYWORDS)

        if is_email_query and all_chunks:
            # Directly filter ALL chunks that came from emails.txt — no similarity search
            email_chunks = [
                c for c in all_chunks
                if "emails.txt" in c.metadata.get("source", "")
```

```python
ef build_qa_chain(vectorstore, local_llm, all_chunks=None):

    def smart_retriever(query):
        query_lower = query.lower()
        is_email_query = any(kw in query_lower for kw in EMAIL_KEYWORDS)

        if is_email_query and all_chunks:
            # Directly filter ALL chunks that came from emails.txt - no similarity search
            email_chunks = [
                c for c in all_chunks
                if "emails.txt" in c.metadata.get("source", "")
            ]
            # Also get semantic results for extra context
            semantic_docs = vectorstore.similarity_search(query, k=3)

            # Merge and deduplicate
            seen = set()
            combined = []
            for doc in email_chunks + semantic_docs:
                key = doc.page_content[:80]
                if key not in seen:
                    seen.add(key)
                    combined.append(doc)
            return combined

        return vectorstore.similarity_search(query, k=5)

    qa_chain = (
        {"context": lambda x: format_docs(smart_retriever(x)), "input": RunnablePassthrough()}
        | prompt
        | local_llm
        | StrOutputParser()
    )

    print("Manual LCEL QA chain created successfully without using langchain.chains")
    return qa_chain
```

3.4 app.py — Streamlit User Interface

This module provides the front-end of the system — the chatbot interface that the user actually sees and interacts with.

Built with Streamlit, the interface displays a clean chat window with a text input field at the bottom. The user can type any question and immediately receive an answer from the chatbot.

To improve performance, the setup() function is decorated with @st.cache_resource, which means the vector database and QA chain are only built once when the app starts, and then reused for every subsequent question. This avoids repeated loading and makes the chatbot fast and responsive.

When the user submits a question, it is passed directly to the QA chain, and the response is displayed in the chat window.

```python
import os
import requests
import streamlit as st
from streamlit_lottie import st_lottie

# 1. Suppress TensorFlow Warnings
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0"
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"

import warnings
warnings.filterwarnings("ignore")

# 2. Logic Imports (Unchanged)
from preprocessing import chunks
from model import local_llm, build_vectorstore
from chatbot import build_qa_chain
from dotenv import load_dotenv

load_dotenv()

# --- PAGE CONFIG ---
st.set_page_config(page_title="FAQ Chatbot", page_icon="💬", layout="centered")

# --- CUSTOM CSS (Sleek monochrome chat) ---
st.markdown("""
    <style>
    .stApp { background-color: #0e1117; color: #ffffff; }

    /* Hide the 'made with streamlit' and other headers */
    #MainMenu {visibility: hidden;}
    footer {visibility: hidden;}
    header {visibility: hidden;}

    /* Sidebar Styling */
    [data-testid="stSidebar"] {
        background-color: #161b22;
        border-right: 1px solid #30363d;
    }

    /* Adjusting chat message styling */
    .stChatMessage {
        background-color: transparent !important;
        padding-top: 1rem;
```

```
        padding-top: 1rem;
    }

    /* Make the assistant bubble slightly different if desired */
    [data-testid="stChatMessage"]:nth-child(even) {
        background-color: #161b22 !important;
        border-radius: 10px;
    }
    </style>
    """, unsafe_allow_html=True)

# --- ANIMATION LOADER ---
def load_lottieurl(url):
    r = requests.get(url)
    return r.json() if r.status_code == 200 else None

lottie_bot = load_lottieurl("https://assets1.lottiefiles.com/packages/lf20_m6cu9sc0.json")

# --- SETUP LOGIC (Cached) ---
@st.cache_resource
def setup():
    vectorstore = build_vectorstore(chunks)
    qa_chain = build_qa_chain(vectorstore, local_llm, all_chunks=chunks)
    return qa_chain

qa_chain = setup()

# --- SIDEBAR ---
with st.sidebar:
    st.image("https://upload.wikimedia.org/wikipedia/commons/7/78/Dropbox_Icon.svg", width=40)
    st.title("Secure FAQ")
    st.markdown("---")
    st.write("• **Engine:** GPT-4o-Mini")
    st.write("• **Source:** Dropbox Docs")
    if st.button("Clear Conversation"):
        st.session_state.messages = []
        st.rerun()
```

```python
# --- CHAT INTERFACE ---

# 1. Header with animation
col1, col2 = st.columns([1, 6])
with col1:
    if lottie_bot:
        st_lottie(lottie_bot, height=60, key="bot")
with col2:
    st.subheader("How can I help you?")

# 2. Initialize Chat History
if "messages" not in st.session_state:
    st.session_state.messages = []

# 3. Display existing messages
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# 4. Chat Input (The GPT-like bar at the bottom)
if prompt := st.chat_input("Type your message here..."):
    # Display user message
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    # Display assistant response
    with st.chat_message("assistant"):
        # We don't use st.success here, just plain text to look like a chat
        placeholder = st.empty()
        with st.spinner(""):
            try:
                response = qa_chain.invoke(prompt)
                placeholder.markdown(response.strip())
                st.session_state.messages.append({"role": "assistant", "content": response.strip()})
            except Exception as e:
                placeholder.error(f"Something went wrong. {str(e)}")
```

4. How the System Works

The following table explains the complete step-by-step flow of the FAQ Chatbot System — from when a user types a question to when they receive a final answer.

Step	Stage	Description
1	**User Input**	The user types a question in the Streamlit chat interface.
2	**Question Encoding**	The question is converted into an embedding (a numerical vector) using the same HuggingFace model used during setup.
3	**Vector Search**	The FAISS database is searched to find the text chunks whose embeddings are most similar (semantically closest) to the question.
4	**Context Building**	The top matching chunks are retrieved and formatted into a single block of context text.
5	**Prompt Assembly**	The system prompt, the context block, and the user's question are combined into a full prompt using the ChatPromptTemplate.
6	**LLM Processing**	The complete prompt is sent to the LLM (via OpenRouter). The LLM reads the context and generates a relevant, accurate answer.
7	**Response Display**	The answer is parsed and displayed in the Streamlit chat window for the user to read.

This entire flow happens in a fraction of a second, making the chatbot feel fast and natural to use. The key advantage of this RAG approach is that the LLM is grounded in the actual document content — it cannot hallucinate answers that are not supported by the uploaded files.

5. Conclusion

This project successfully demonstrates how modern AI technologies can be combined to build a practical, document-aware FAQ chatbot. By using LangChain's modular components, the system handles the full pipeline from raw document loading to intelligent answer generation.

The chatbot is capable of reading multiple types of documents, understanding their content through vector embeddings, and answering user questions accurately using a state-of-the-art language model. The Streamlit interface makes the system accessible and easy to use for anyone, without requiring any technical knowledge.

Overall, this project provides a strong foundation for building production-ready customer support systems, knowledge bases, and document-driven AI assistants. With improvements such as adding more document sources, fine-tuning the retrieval logic, or expanding the user interface, this system can be scaled into a fully enterprise-grade FAQ solution.

Key Achievements

- Multi-format document ingestion (PDF + TXT)
- Semantic search using FAISS vector database
- Context-aware answer generation via LLM
- Interactive web interface powered by Streamlit
- Modular, maintainable code architecture